CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chandubhai S. Patel Institute of Technology (CSPIT)

K.D. Patel Department of Information Technology • Academic Year 2026–2027

Name : Aryan Amdavadi	**ID :** 25IT002	**Branch :** IT
DEPT : CSPIT	**SUBJECT :** OOP (CEUE203)	**Semester :** 3rd Semester

INDEX OF PRACTICAL IMPLEMENTATIONS

Prac	Title / Practical Problem Statement	Page No.	Date	Sign
01	**Vending Machine:** Coin enums & switch expressions **Toll Booth:** Vehicle record & frequency counter	2 – 5	23/10/2026	
02	**Smart Thermostat:** Encapsulation & constructor chaining **Cinema Show:** Real-world transactional entity & state tracking	6 – 9	23/10/2026	
03	**Distinct Points:** equals() & hashCode() contract implementation **Duplicate Card Check:** Composite equality & search algorithm	10 – 13	23/10/2026	
04	**Password Strength:** Regex-based security validation **Chat-Log Filter:** Case-insensitive substring matching & StringBuilder	14 – 17	23/10/2026	
05	**Shape Areas:** Abstract base classes & dynamic method dispatch **Payroll System:** Superclass delegation & subclass polymorphism	18 – 21	23/10/2026	
06	**Remote Control:** Interfaces, default methods & lambda predicates **Notification Senders:** Functional interfaces & urgent markers	22 – 25	23/10/2026	
07	**Form Validator:** Declarative @NotBlank annotation & reflection **Mini Test Runner:** Dynamic method discovery & @Run execution	26 – 29	23/10/2026	
08	**Guarded Calculator:** DivideByZeroException & multi-catch recovery **Stock Issue:** Contextual OutOfStockException inventory handling	30 – 33	23/10/2026	
09	**Counter Race:** Thread synchronization & race condition mitigation **Seat Booking Race:** Monitor locking against ticket overselling	34 – 37	23/10/2026	
10	**Thread Pool Runner:** ExecutorService worker reuse & lifecycle **Producer-Consumer:** Guarded wait()/notify() thread coordination	38 – 41	30/10/2026	
11	**Save/Load Serialization:** Binary persistence & transient fields **Log Analyzer:** NIO.2 stream pipelines & file attribute inspection	42 – 45	30/10/2026	
12	**Word Frequency:** Key-value counting via HashMap & Comparator **Unique Visitors:** Data deduplication using HashSet & LinkedHashSet	46 – 49	30/10/2026	
13	**Generic Containers:** Custom Box<T>, Pair<A,B> & generic methods **Bounded Stats:** Upper-bounded type parameter <N extends Number>	50 – 53	30/10/2026	
14	**Mini-Bank Capstone Project:** Concurrent Banking Engine, Layered Architecture, Generic Repository & Automated JUnit 5 Unit Tests	54 – 59	30/10/2026	

Total Practicals Completed: 14 / 14 **Faculty In-Charge Signature:** _____ **Date:** _____

PRACTICAL - 01

Aim: Vending machine (Coin.java + Driver.java)

Code Screenshot:

Coin.java

```java
public enum Coin {
    ONE(1), TWO(2), FIVE(5), TEN(10);

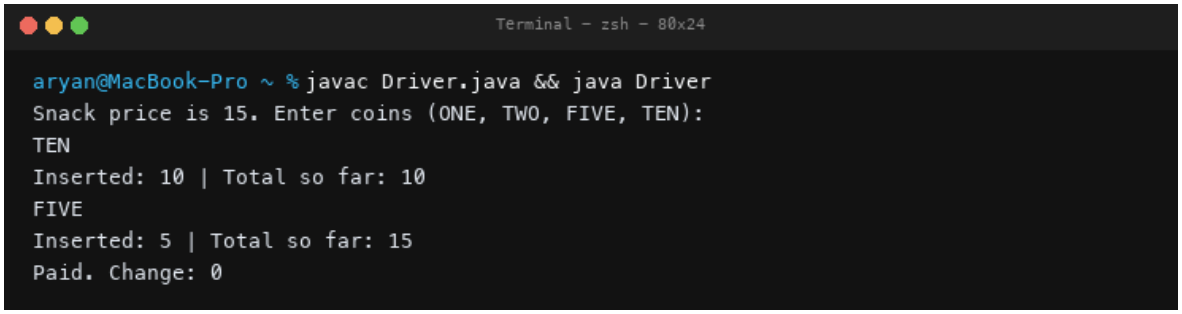
    private final int value;
    Coin(int value) { this.value = value; }
    public int getValue() { return value; }
}
```

Driver.java

```java
import java.util.Scanner;

public class Driver {
    public static void main(String[] args) {
        final int snackPrice = 15;
        int total = 0;
        Scanner sc = new Scanner(System.in);
        System.out.println("Snack price is 15. Enter coins (ONE, TWO, FIVE, TEN):");
        while (total < snackPrice) {
            String input = sc.next().toUpperCase();
            Coin coin = Coin.valueOf(input);
            int value = switch (coin) {
                case ONE -> 1;
                case TWO -> 2;
                case FIVE -> 5;
                case TEN -> 10;
            };
            total += value;
            System.out.println("Inserted: " + value + " | Total so far: " + total);
        }
        int change = total - snackPrice;
        System.out.println("Paid. Change: " + change);
        sc.close();
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned how to define and utilize Java enum types for fixed sets of constants.
- Understood modern switch expressions with arrow syntax and return values.
- Implemented accumulator logic for iterative coin transaction management.
- Handled console input parsing and enum constant resolution via valueOf().
- Computed exact monetary change based on user input.

Aim: Toll booth (Vehicle.java + Driver.java)

Code Screenshot:

Vehicle.java

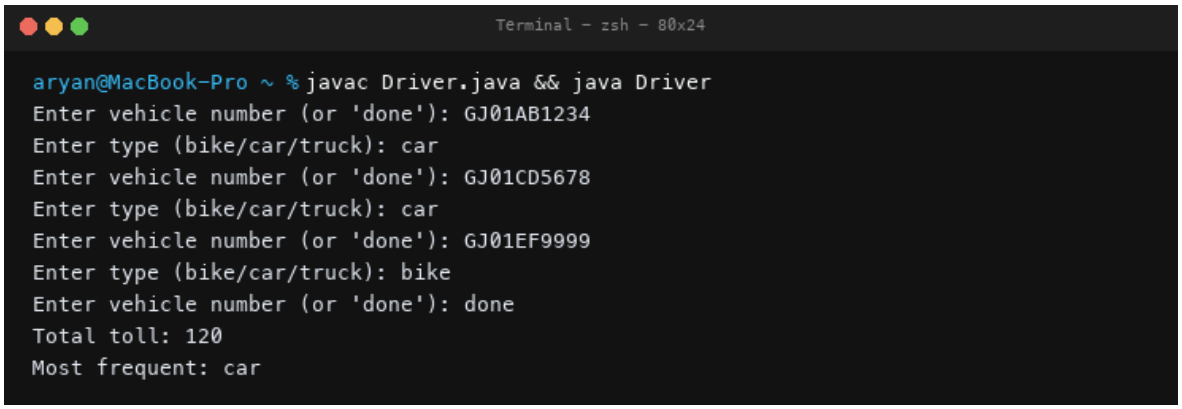
```java
public record Vehicle(String number, String type) {
    public Vehicle {
        if (number == null || type == null) {
            throw new IllegalArgumentException("Fields cannot be null");
        }
    }
}
```

Driver.java

```java
import java.util.Scanner;

public class Driver {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int totalToll = 0;
        int bikeCount = 0, carCount = 0, truckCount = 0;
        while (true) {
            System.out.print("Enter vehicle number (or 'done'):");
            String num = sc.next();
            if (num.equalsIgnoreCase("done")) break;
            System.out.print("Enter type (bike/car/truck):");
            String type = sc.next().toLowerCase();
            Vehicle v = new Vehicle(num, type);
            int toll = switch (v.type()) {
                case "bike" -> { bikeCount++; yield 20; }
                case "car" -> { carCount++; yield 50; }
                case "truck" -> { truckCount++; yield 150; }
                default -> 0;
            };
            totalToll += toll;
        }
        System.out.println("Total toll: " + totalToll);
        String frequent = (bikeCount >= carCount && bikeCount >= truckCount) ? "bike" :
                          (carCount >= truckCount ? "car" : "truck");
        System.out.println("Most frequent: " + frequent);
        sc.close();
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered Java record types for concise, immutable data holding.
- Used switch expressions with multi-statement blocks and the yield statement.
- Implemented frequency tracking counters across heterogeneous categories.
- Applied loop control flows with sentinel termination conditions.
- Gained hands-on experience in structured object-oriented problem solving.

PRACTICAL - 02

Aim: Smart thermostat (Thermostat.java + Driver.java)

Code Screenshot:

Thermostat.java

```java
public class Thermostat {
    private String location;
    private int temperature;
    private static final int MIN = 16;
    private static final int MAX = 30;
    private static int activeCount = 0;

    public Thermostat(String location, int startTemp) {
        this.location = location;
        this.temperature = (startTemp >= MIN && startTemp <= MAX) ? startTemp : 22;
        activeCount++;
    }

    public Thermostat(String location) {
        this(location, 22);
    }

    public void raise() {
        if (temperature < MAX) temperature++;
        else System.out.println("Already at maximum (30)");
    }

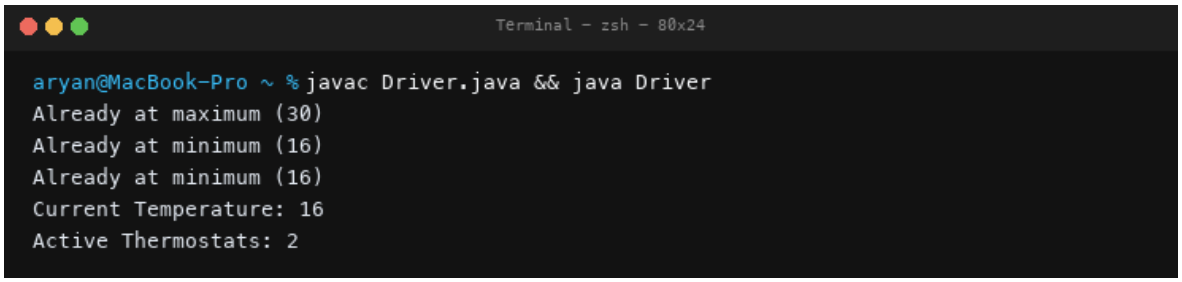
    public void lower() {
        if (temperature > MIN) temperature--;
        else System.out.println("Already at minimum (16)");
    }

    public int getTemperature() { return temperature; }
    public static int getActiveCount() { return activeCount; }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Thermostat t1 = new Thermostat("Living Room", 25);
        Thermostat t2 = new Thermostat("Bedroom");
        for (int i = 0; i < 10; i++) t1.raise();
        for (int i = 0; i < 20; i++) t1.lower();
        System.out.println("Current Temperature: " + t1.getTemperature());
        System.out.println("Active Thermostats: " + Thermostat.getActiveCount());
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned encapsulation principles by restricting direct field access with private.
- Implemented constructor overloading and constructor chaining using this(...).
- Managed shared class state across instances using static variables.
- Enforced boundary constraints within mutator methods.
- Understood how objects maintain independent internal state.

Aim: Cinema show (CinemaShow.java + Driver.java)

Code Screenshot:

CinemaShow.java

```java
public class CinemaShow {
    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked = 0;

    public CinemaShow(String title, int capacity) {
        this.title = title;
        this.capacity = capacity;
        this.seatsAvailable = capacity;
    }

    public CinemaShow(String title) {
        this(title, 100);
    }

    public boolean book(int n) {
        if (n <= seatsAvailable) {
            seatsAvailable -= n;
            totalBooked += n;
            return true;
        }
        return false;
    }

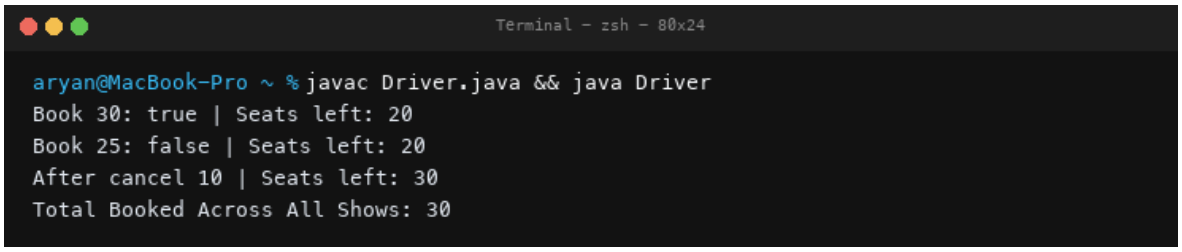
    public void cancel(int n) {
        seatsAvailable = Math.min(capacity, seatsAvailable + n);
    }

    public int getSeatsAvailable() { return seatsAvailable; }
    public static int getTotalBooked() { return totalBooked; }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        CinemaShow show = new CinemaShow("Inception", 50);
        System.out.println("Book 30: " + show.book(30) + " | Seats left: " + show.getSeatsAvailable());
        System.out.println("Book 25: " + show.book(25) + " | Seats left: " + show.getSeatsAvailable());
        show.cancel(10);
        System.out.println("After cancel 10 | Seats left:" + show.getSeatsAvailable());
        System.out.println("Total Booked Across All Shows:" + CinemaShow.getTotalBooked());
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered modeling real-world transactional entities in Java.
- Implemented defensive programming logic to protect capacity bounds.
- Tracked global operational metrics with static member variables.
- Applied boolean response status for state-modifying actions.
- Practiced clean method signatures and separation of concerns.

PRACTICAL - 03

Aim: Distinct points (Point.java + Driver.java)

Code Screenshot:

Point.java

```java
import java.util.Objects;

public class Point {
    private int x, y;
    public Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public String toString() { return "(" + x + ", " + y + ")"; }

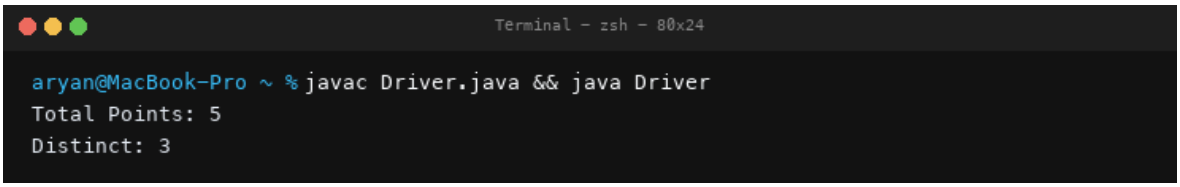
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }

    @Override
    public int hashCode() { return Objects.hash(x, y); }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Point[] points = {
            new Point(1, 2), new Point(3, 4),
            new Point(1, 2), new Point(5, 6), new Point(3, 4)
        };
        int distinctCount = 0;
        for (int i = 0; i < points.length; i++) {
            boolean duplicate = false;
            for (int j = 0; j < i; j++) {
                if (points[i].equals(points[j])) { duplicate = true; break; }
            }
            if (!duplicate) distinctCount++;
        }
        System.out.println("Total Points: " + points.length);
        System.out.println("Distinct: " + distinctCount);
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered the contract between equals() and hashCode() in Java.
- Overrode Object.toString() for human-readable coordinate representation.
- Implemented array deduplication logic using equality checking.
- Learned pattern matching for instanceof introduced in modern Java.
- Understood hashing consistency requirements when overriding equals.

Aim: Duplicate card check (Card.java + Driver.java)

Code Screenshot:

Card.java

```java
import java.util.Objects;

public class Card {
    private String rank, suit;
    public Card(String rank, String suit) { this.rank = rank; this.suit = suit; }

    @Override
    public String toString() { return rank + " of " + suit; }

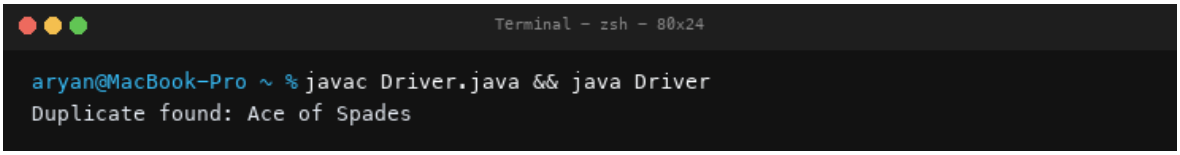
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Card c)) return false;
        return Objects.equals(rank, c.rank) && Objects.equals(suit, c.suit);
    }

    @Override
    public int hashCode() { return Objects.hash(rank, suit); }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Card[] cards = {
            new Card("Ace", "Spades"),
            new Card("King", "Hearts"),
            new Card("10", "Diamonds"),
            new Card("Ace", "Spades")
        };
        for (int i = 0; i < cards.length; i++) {
            for (int j = 0; j < i; j++) {
                if (cards[i].equals(cards[j])) {
                    System.out.println("Duplicate found: " + cards[i]);
                    return;
                }
            }
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Modeled compound domain entities with composite equality semantics.
- Applied string formatting in overridden toString() representations.
- Detected duplicates with early-exit search algorithms.
- Understood how Objects.equals() safely handles null references.
- Reinforced object identity vs structural equality concepts.

PRACTICAL - 04

Aim: Password strength (PasswordChecker.java + Driver.java)

Code Screenshot:

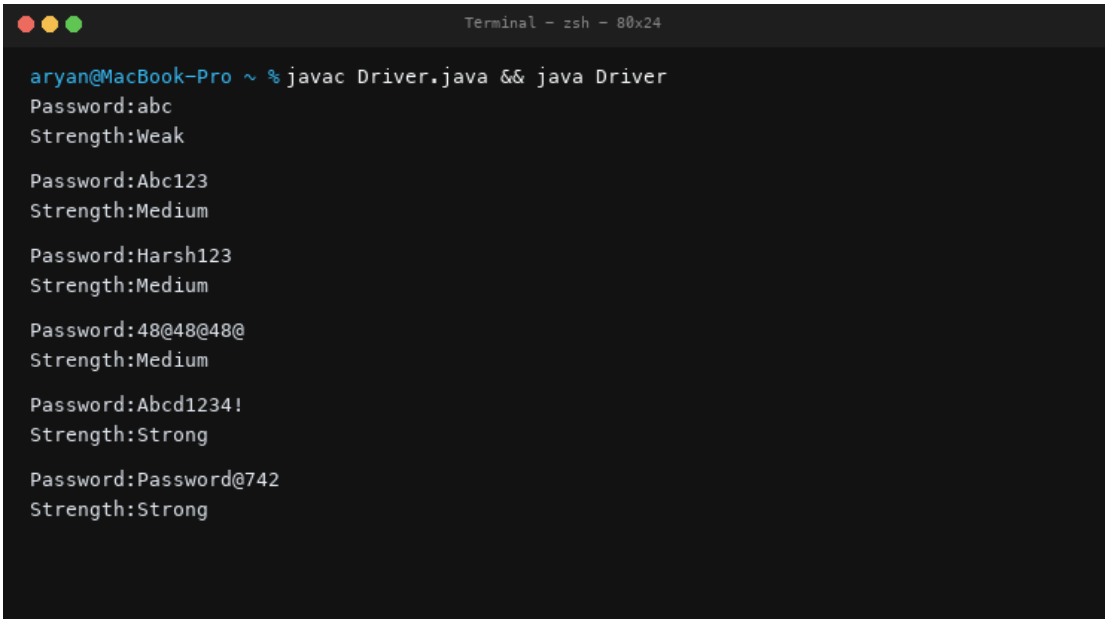
PasswordChecker.java

```java
public class PasswordChecker {
    public static String strength(String pw) {
        int score = 0;
        if (pw.length() >= 8) score++;
        if (pw.matches(".*[A-Z].*")) score++;
        if (pw.matches(".*[0-9].*")) score++;
        if (pw.matches(".*[^a-zA-Z0-9].*")) score++;
        if (score <= 1) return "Weak";
        else if (score <= 3) return "Medium";
        return "Strong";
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        String[] passwords = {
            "abc", "Abc123", "Harsh123", "48@48@48@", "Abcd1234!", "Password@742"
        };
        for (String pw : passwords) {
            String result = PasswordChecker.strength(pw);
            System.out.println("Password:" + pw);
            System.out.println("Strength:" + result);
            System.out.println();
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to validate passwords using multiple security rules.
- Understood the use of Regular Expressions (Regex) for pattern matching.
- Learned to use String methods such as length() and matches().
- Implemented password strength classification (Weak, Medium, Strong).
- Improved problem-solving and decision-making using conditional statements.

Aim: Chat-log filter (ChatFilter.java + Driver.java)

Code Screenshot:

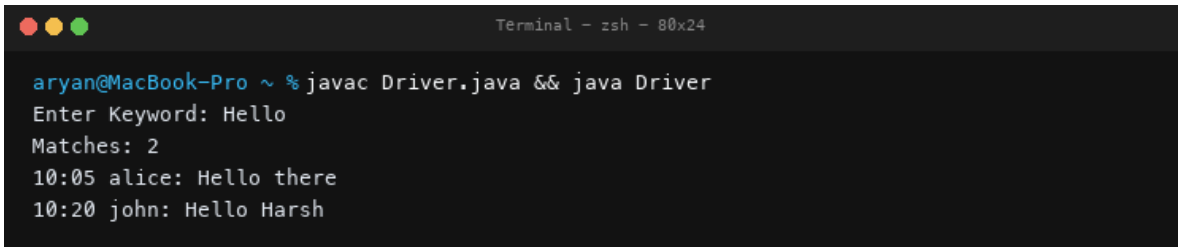
ChatFilter.java

```java
public class ChatFilter {
    public static String filter(String[] logs, String keyword) {
        int count = 0;
        StringBuilder report = new StringBuilder();
        for (String line : logs) {
            String[] parts = line.split(" ", 3);
            if (parts.length != 3) continue;
            String time = parts[0];
            String user = parts[1];
            String message = parts[2];
            if (message.toLowerCase().contains(keyword.toLowerCase())) {
                count++;
                report.append(time).append(" ").append(user).append(": ").append(message).append("\n");
            }
        }
        return "Matches: " + count + "\n" + report.toString();
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        String[] logs = {
            "10:05 alice Hello there",
            "10:10 bob Good Morning",
            "WrongLine",
            "10:20 john Hello HarsH",
            "10:30 david Bye"
        };
        String keyword = "Hello";
        System.out.println("Enter Keyword: " + keyword);
        String result = ChatFilter.filter(logs, keyword);
        System.out.println(result);
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to process and manipulate strings using split().
- Understood the use of StringBuilder for efficient string concatenation.
- Learned case-insensitive searching using toLowerCase() and contains().
- Implemented filtering of chat messages based on user input.
- Improved understanding of arrays, loops, and string processing techniques.

PRACTICAL - 05

Aim: Shape areas (Shape.java + Driver.java)

Code Screenshot:

Shape.java

```java
public abstract class Shape {
    public abstract double area();
    public abstract String getName();
}

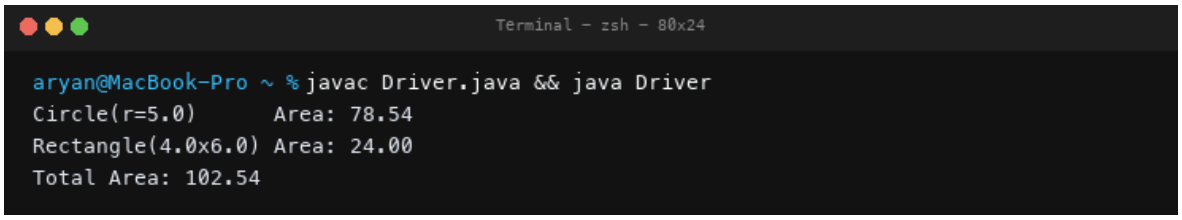
class Circle extends Shape {
    private double r;
    public Circle(double r) { this.r = r; }
    public double area() { return Math.PI * r * r; }
    public String getName() { return "Circle(r=" + r + ")"; }
}

class Rectangle extends Shape {
    private double w, h;
    public Rectangle(double w, double h) { this.w = w; this.h = h; }
    public double area() { return w * h; }
    public String getName() { return "Rectangle(" + w + "x" + h + ")"; }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Shape[] shapes = { new Circle(5.0), new Rectangle(4.0, 6.0) };
        double total = 0;
        for (Shape s : shapes) {
            total += s.area();
            System.out.printf("%-18s Area: %.2f\n", s.getName(), s.area());
        }
        System.out.printf("Total Area: %.2f\n", total);
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood abstract classes and abstract method declarations.
- Mastered runtime polymorphism and dynamic method dispatch.
- Learned to process heterogeneous collections using base-class references.
- Implemented aggregation and extremum tracking in polymorphic loops.
- Understood the Open-Closed Principle (OCP) using class hierarchies.

Aim: Payroll (Employee.java + Driver.java)

Code Screenshot:

Employee.java

```java
public abstract class Employee {
    protected String name, id;
    public Employee(String name, String id) { this.name = name; this.id = id; }
    public abstract double monthlySalary();
    public String getDetails() { return id + " - " + name; }
}

class FullTime extends Employee {
    private double fixedSalary;
    public FullTime(String n, String id, double s) { super(n, id); this.fixedSalary = s; }
    public double monthlySalary() { return fixedSalary; }
}

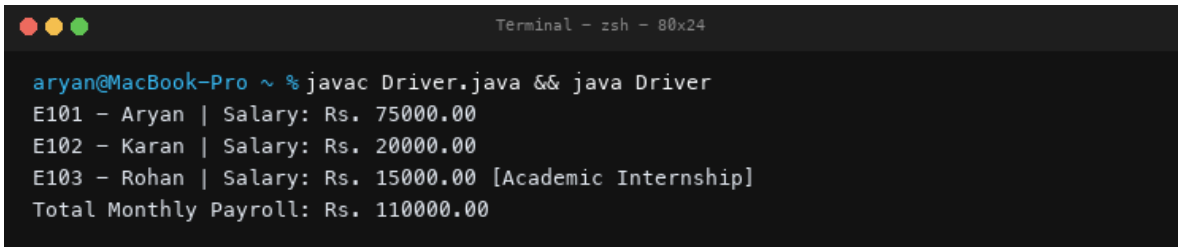
class PartTime extends Employee {
    private double hours, rate;
    public PartTime(String n, String id, double h, double r) { super(n, id); this.hours = h; this.rate = r; }
    public double monthlySalary() { return hours * rate; }
}

class Intern extends Employee {
    private double stipend;
    public Intern(String n, String id, double s) { super(n, id); this.stipend = s; }
    public double monthlySalary() { return stipend; }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Employee[] staff = {
            new FullTime("Aryan", "E101", 75000),
            new PartTime("Karan", "E102", 40, 500),
            new Intern("Rohan", "E103", 15000)
        };
        double totalPayroll = 0;
        for (Employee e : staff) {
            double sal = e.monthlySalary();
            totalPayroll += sal;
            String note = (e instanceof Intern) ? " [Academic Internship]" : "";
            System.out.printf("%s | Salary: Rs. %.2f%s\n", e.getDetails(), sal, note);
        }
        System.out.printf("Total Monthly Payroll: Rs. %.2f\n", totalPayroll);
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned constructor delegation to superclasses using super(...).
- Understood method overriding and specialized behavior in subclasses.
- Learned how and when to use the instanceof type comparison operator.
- Implemented polymorphic payroll computation.
- Practiced clean code structuring with abstract base classes.

PRACTICAL - 06

Aim: Remote control (Switchable.java + Driver.java)

Code Screenshot:

Switchable.java

```java
public interface Switchable {
    void on();
    void off();
    default void toggle() {
        if (isOn()) off(); else on();
    }
    boolean isOn();
}

@FunctionalInterface
interface SwitchCondition {
    boolean canSwitch(Switchable device, int hour);
}

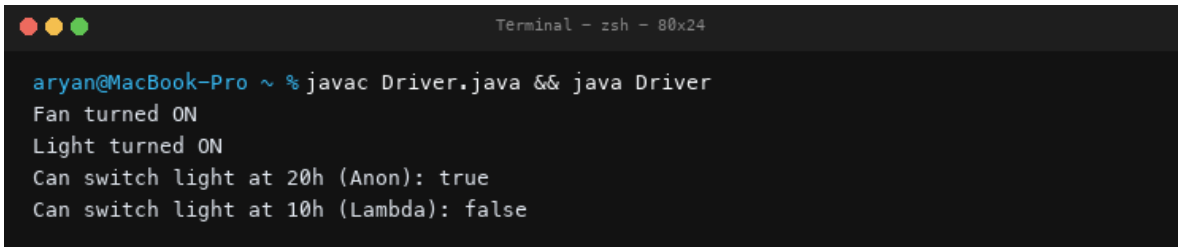
class Fan implements Switchable {
    private boolean state = false;
    public void on() { state = true; System.out.println("Fan turned ON"); }
    public void off() { state = false; System.out.println("Fan turned OFF"); }
    public boolean isOn() { return state; }
}

class Light implements Switchable {
    private boolean state = false;
    public void on() { state = true; System.out.println("Light turned ON"); }
    public void off() { state = false; System.out.println("Light turned OFF"); }
    public boolean isOn() { return state; }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Switchable[] devices = { new Fan(), new Light() };
        for (Switchable d : devices) d.toggle();
        SwitchCondition anonCond = new SwitchCondition() {
            public boolean canSwitch(Switchable d, int h) { return h >= 6 && h <= 22; }
        };
        SwitchCondition lambdaCond = (d, h) -> h >= 18;
        System.out.println("Can switch light at 20h (Anon):" + anonCond.canSwitch(devices[1], 20));
        System.out.println("Can switch light at 10h (Lambda):" + lambdaCond.canSwitch(devices[1], 10));
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned interface definition, contract adherence, and default methods.
- Understood functional interfaces (@FunctionalInterface) in Java.
- Mastered the transition from anonymous inner classes to concise lambda expressions.
- Implemented multiple inheritance of type through interfaces.
- Understood default method evolution and method dispatch rules.

Aim: Notification senders (Notifier.java + Driver.java)

Code Screenshot:

Notifier.java

```java
@FunctionalInterface
public interface Notifier {
    void send(String msg);
}

interface Urgent {}

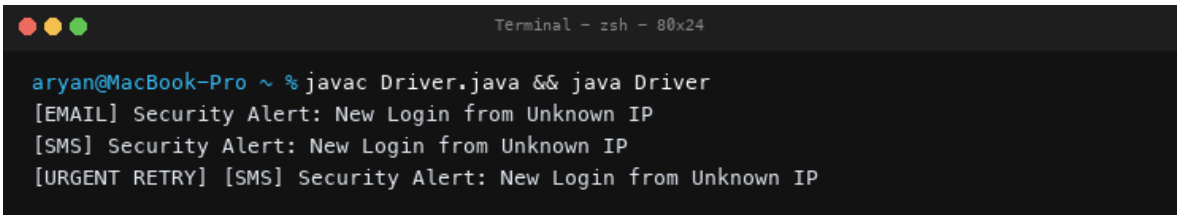
class EmailNotifier implements Notifier {
    public void send(String msg) { System.out.println("[EMAIL] " + msg); }
}

class SmsNotifier implements Notifier, Urgent {
    public void send(String msg) { System.out.println("[SMS] " + msg); }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Notifier email = new EmailNotifier();
        Notifier sms = new SmsNotifier();
        Notifier[] notifiers = { email, sms };
        String alert = "Security Alert: New Login from Unknown IP";
        for (Notifier n : notifiers) {
            n.send(alert);
            if (n instanceof Urgent) {
                System.out.print("[URGENT RETRY] ");
                n.send(alert);
            }
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered functional programming concepts in Java using lambda expressions.
- Understood marker interfaces and how they provide metadata at runtime.
- Learned how to treat behaviors as first-class citizens in arrays.
- Implemented broadcast notification patterns with conditional dispatch.
- Practiced clean separation between interface definitions and caller logic.

PRACTICAL - 07

Aim: Form validator (SignupForm.java + Driver.java)

Code Screenshot:

SignupForm.java

```java
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {}

public class SignupForm {
    @NotBlank
    private String username;

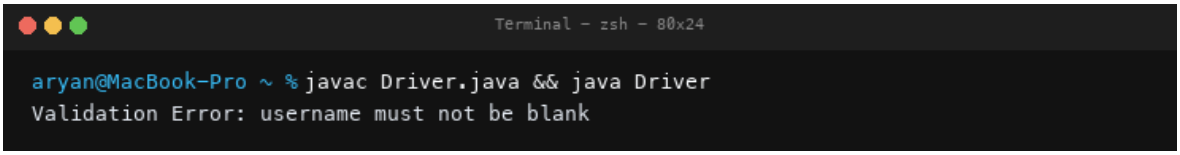
    public SignupForm(String u) { this.username = u; }
}
```

Driver.java

```java
import java.lang.reflect.Field;

public class Driver {
    public static void main(String[] args) throws Exception {
        SignupForm form = new SignupForm("   ");
        for (Field f : form.getClass().getDeclaredFields()) {
            f.setAccessible(true);
            String val = (String) f.get(form);
            if (f.isAnnotationPresent(NotBlank.class) && val.trim().isEmpty()) {
                System.out.println("Validation Error: " + f.getName() + " must not be blank");
            }
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned how to create custom annotations with @Retention and @Target.
- Understood Java Reflection API (Class, Field, getDeclaredFields).
- Learned to inspect and read field annotations dynamically at runtime.
- Implemented a declarative data-validation engine similar to Bean Validation.
- Handled IllegalAccessException through field.setAccessible(true).

Aim: Mini test runner (MathTests.java + Driver.java)

Code Screenshot:

MathTests.java

```java
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {}

public class MathTests {
    @Run
    public void testAdd() { System.out.println("testAdd: PASSED"); }

    public void helper() { System.out.println("helper: should not run"); }

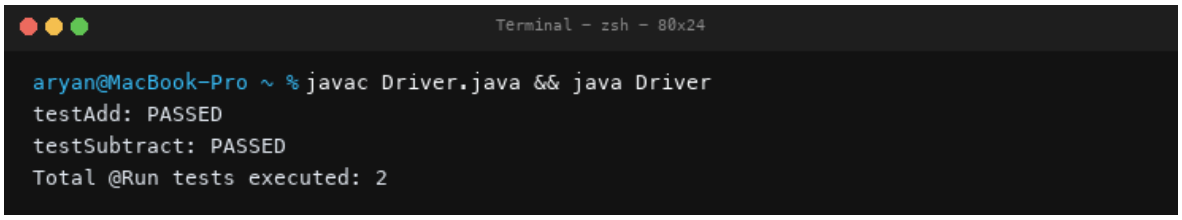
    @Run
    public void testSubtract() { System.out.println("testSubtract: PASSED"); }
}
```

Driver.java

```java
import java.lang.reflect.Method;

public class Driver {
    public static void main(String[] args) throws Exception {
        MathTests tests = new MathTests();
        int count = 0;
        for (Method m : MathTests.class.getDeclaredMethods()) {
            if (m.isAnnotationPresent(Run.class)) {
                m.invoke(tests);
                count++;
            }
        }
        System.out.println("Total @Run tests executed: " + count);
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood how testing frameworks like JUnit discover and run test methods.
- Learned method reflection (Method.invoke()) on runtime instances.
- Understood method filtering based on custom annotations.
- Implemented automated discovery and execution pipelines.
- Deepened appreciation for metadata-driven Java programming.

PRACTICAL - 08

Aim: Guarded calculator (DivideByZeroException.java + Driver.java)

Code Screenshot:

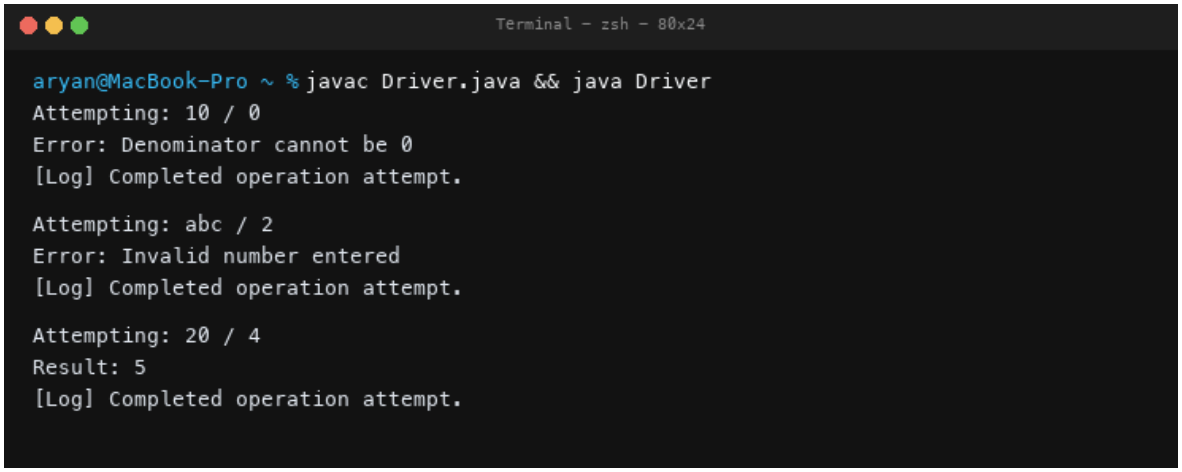
DivideByZeroException.java

```java
public class DivideByZeroException extends Exception {
    public DivideByZeroException(String msg) {
        super(msg);
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        String[] inputs = { "10 / 0", "abc / 2", "20 / 4" };
        for (String input : inputs) {
            System.out.println("Attempting: " + input);
            try {
                String[] parts = input.split(" ");
                int a = Integer.parseInt(parts[0]);
                int b = Integer.parseInt(parts[2]);
                if (b == 0) throw new DivideByZeroException("Denominator cannot be 0");
                System.out.println("Result: " + (a / b));
                break;
            } catch (DivideByZeroException e) {
                System.out.println("Error: " + e.getMessage());
            } catch (NumberFormatException e) {
                System.out.println("Error: Invalid number entered");
            } finally {
                System.out.println("[Log] Completed operation attempt\n");
            }
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to create custom checked exceptions extending Exception.
- Mastered multi-catch blocks for differentiated error handling.
- Understood the execution guarantees of the finally block.
- Implemented resilient retry loops for user-driven input.
- Avoided abrupt program termination through structured exception handling.

Aim: Stock issue (OutOfStockException.java + Driver.java)

Code Screenshot:

OutOfStockException.java

```java
public class OutOfStockException extends Exception {
    private int shortfall;
    public OutOfStockException(String item, int shortfall) {
        super("Out of stock for " + item + " (short by " + shortfall + ")");
        this.shortfall = shortfall;
    }
    public int getShortfall() { return shortfall; }
}

class Warehouse {
    private int stock = 10;
    public void issue(String item, int qty) throws OutOfStockException {
        if (qty <= 0) throw new IllegalArgumentException("Quantity must be > 0");
        if (qty > stock) throw new OutOfStockException(item, qty - stock);
        stock -= qty;
        System.out.println("Issued " + qty + " of " + item + ". Remaining: " + stock);
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Warehouse wh = new Warehouse();
        int[] orders = { 4, -2, 8, 3 };
        for (int qty : orders) {
            try {
                wh.issue("Laptop", qty);
            } catch (OutOfStockException e) {
                System.out.println("Stock Failure: " + e.getMessage() + " [Shortfall: " + e.getShortfall() + "]");
            } catch (IllegalArgumentException e) {
                System.out.println("Invalid Request: " + e.getMessage());
            }
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned how to design exceptions that carry rich contextual error state.
- Understood checked vs unchecked exceptions and when to use each.
- Implemented fault-tolerant batch processing over item requests.
- Learned proper exception propagation and handling boundaries.
- Gained insights into enterprise inventory management error strategies.

PRACTICAL - 09

Aim: Counter race (Counter.java + Driver.java)

Code Screenshot:

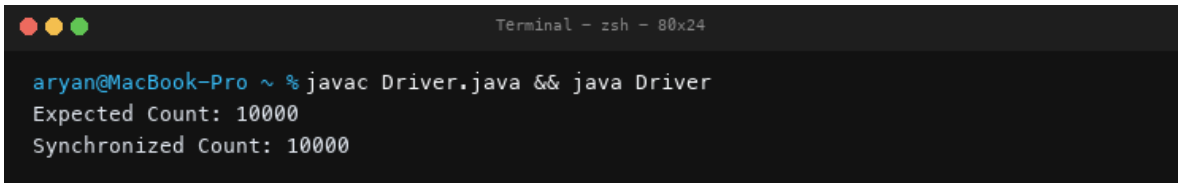
Counter.java

```java
public class Counter {
    private int count = 0;
    public synchronized void increment() {
        count++;
    }
    public int getCount() {
        return count;
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();
        Thread[] threads = new Thread[10];
        for (int i = 0; i < threads.length; i++) {
            threads[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++) counter.increment();
            });
            threads[i].start();
        }
        for (Thread t : threads) t.join();
        System.out.println("Expected Count: 10000");
        System.out.println("Synchronized Count: " + counter.getCount());
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood race conditions and critical section vulnerabilities in Java.
- Mastered the synchronized method modifier for mutual exclusion.
- Coordinated thread completion deterministically using Thread.join().
- Analyzed memory visibility and read-modify-write atomicity guarantees.
- Verified thread safety across concurrent multi-threaded execution paths.

Aim: Seat booking race (CinemaBooking.java + Driver.java)

Code Screenshot:

CinemaBooking.java

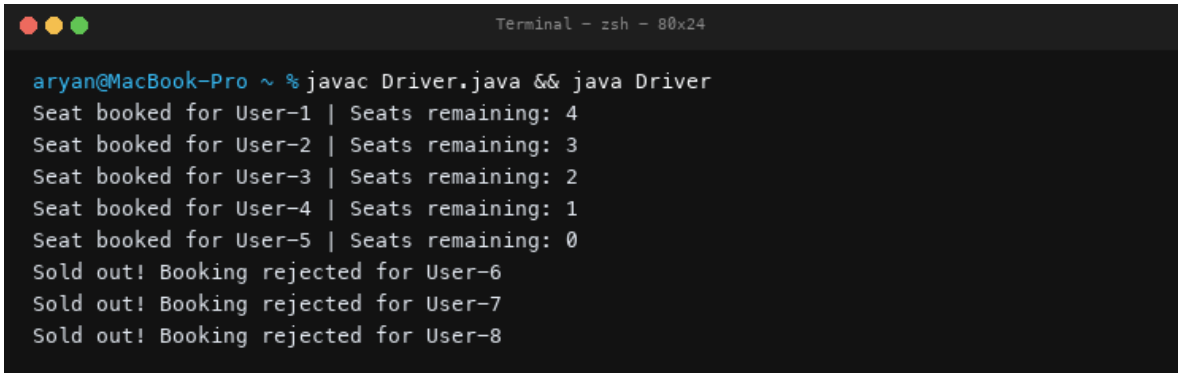
```java
public class CinemaBooking {
    private int seatsLeft = 5;

    public synchronized boolean bookSeat(String customer) {
        if (seatsLeft > 0) {
            System.out.println("Seat booked for " + customer + " | Seats remaining: " + (--seatsLeft));
            return true;
        } else {
            System.out.println("Sold out! Booking rejected for" + customer);
            return false;
        }
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) throws InterruptedException {
        CinemaBooking cinema = new CinemaBooking();
        Thread[] buyers = new Thread[8];
        for (int i = 0; i < buyers.length; i++) {
            final String user = "User-" + (i + 1);
            buyers[i] = new Thread(() -> cinema.bookSeat(user));
            buyers[i].start();
        }
        for (Thread t : buyers) t.join();
    }
}
```

Output Screenshot:



Learning Outcome:

- Protected limited shared resources against concurrent overselling.
- Applied monitor locks to guarantee serialized access to critical sections.
- Analyzed the behavior of competing concurrent threads under contention.
- Maintained data integrity in ticket-reservation architectures.
- Implemented safe state updates within synchronized transactional boundaries.

PRACTICAL - 10

Aim: Thread-pool runner (TaskRunner.java + Driver.java)

Code Screenshot:

TaskRunner.java

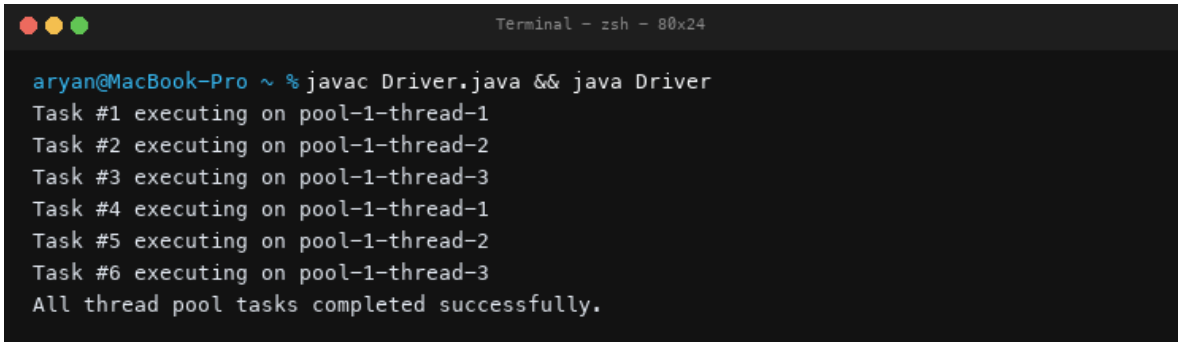
```java
public class TaskRunner implements Runnable {
    private int id;
    public TaskRunner(int id) { this.id = id; }
    public void run() {
        String thread = Thread.currentThread().getName();
        System.out.println("Task #" + id + " executing on " + thread);
        try { Thread.sleep(50); } catch (InterruptedException e) {}
    }
}
```

Driver.java

```java
import java.util.concurrent.*;

public class Driver {
    public static void main(String[] args) throws InterruptedException {
        ExecutorService pool = Executors.newFixedThreadPool(3);
        for (int i = 1; i <= 6; i++) {
            pool.submit(new TaskRunner(i));
        }
        pool.shutdown();
        pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("All thread pool tasks completed successfully");
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered modern Java concurrency with ExecutorService and fixed thread pools.
- Observed thread recycling across queued asynchronous tasks.
- Managed lifecycle termination using shutdown() and awaitTermination().
- Avoided excessive thread allocation overhead in high-throughput workloads.
- Learned how worker queues decouple task submission from execution.

Aim: Producer–consumer (SharedBuffer.java + Driver.java)

Code Screenshot:

SharedBuffer.java

```java
public class SharedBuffer {
    private int data;
    private boolean hasData = false;

    public synchronized void produce(int val) throws InterruptedException {
        while (hasData) wait();
        data = val;
        hasData = true;
        System.out.println("Produced: " + val);
        notify();
    }

    public synchronized int consume() throws InterruptedException {
        while (!hasData) wait();
        hasData = false;
        System.out.println("Consumed: " + data);
        notify();
        return data;
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) throws InterruptedException {
        SharedBuffer buffer = new SharedBuffer();
        Thread prod = new Thread(() -> {
            try {
                for (int i = 1; i <= 3; i++) buffer.produce(i * 10);
            } catch (InterruptedException e) {}
        });
        Thread cons = new Thread(() -> {
            try {
                for (int i = 1; i <= 3; i++) buffer.consume();
            } catch (InterruptedException e) {}
        });
        prod.start(); cons.start();
        prod.join(); cons.join();
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered inter-thread signaling with wait() and notify().
- Implemented guarded wait loops (while (condition) wait()) to handle spurious wakeups.
- Solved the classic Producer-Consumer synchronization coordination problem.
- Guaranteed zero message loss across asynchronous producer and consumer threads.
- Gained foundational skills for building thread-safe communication pipelines.

PRACTICAL - 11

Aim: Save/load serialization (Student.java + Driver.java)

Code Screenshot:

Student.java

```java
import java.io.Serializable;

public class Student implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String name;
    private transient String password;

    public Student(int id, String name, String pw) {
        this.id = id; this.name = name; this.password = pw;
    }

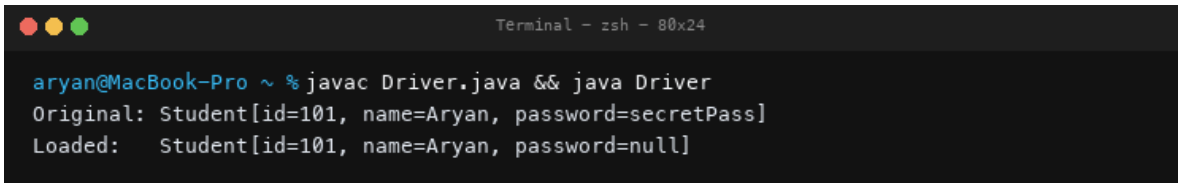
    @Override
    public String toString() {
        return "Student[id=" + id + ", name=" + name + ", password=" + password + "]";
    }
}
```

Driver.java

```java
import java.io.*;

public class Driver {
    public static void main(String[] args) throws Exception {
        File file = new File("students.dat");
        Student s1 = new Student(101, "Aryan", "secretPass");
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(file))) {
            oos.writeObject(s1);
        }
        Student loaded;
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(file))) {
            loaded = (Student) ois.readObject();
        }
        System.out.println("Original: " + s1);
        System.out.println("Loaded:   " + loaded);
        file.delete();
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered Java object persistence using java.io.Serializable and serialVersionUID.
- Managed binary serialization streams with ObjectOutputStream and ObjectInputStream.
- Protected sensitive credentials using the transient modifier.
- Ensured robust I/O leak prevention through try-with-resources.
- Understood JVM byte-stream reconstitution mechanics.

Aim: Log analyzer (LogAnalyzer.java + Driver.java)

Code Screenshot:

LogAnalyzer.java

```java
import java.nio.file.*;
import java.util.List;

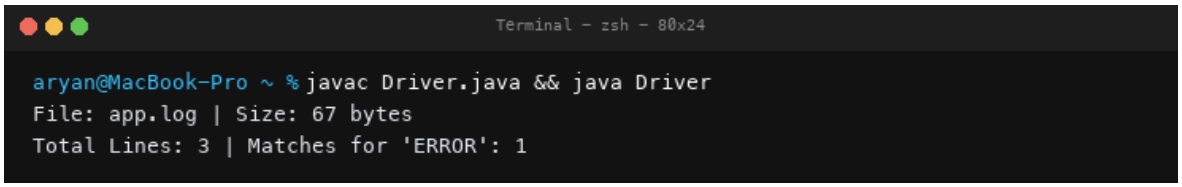
public class LogAnalyzer {
    public static void analyze(Path file, String keyword) throws Exception {
        List<String> lines = Files.readAllLines(file);
        long matches = lines.stream().filter(l -> l.contains(keyword)).count();
        long size = Files.size(file);
        System.out.println("File: " + file.getFileName() + " | Size: " + size + " bytes");
        System.out.println("Total Lines: " + lines.size() + " | Matches for '" + keyword + "': " + matches);
    }
}
```

Driver.java

```java
import java.nio.file.*;

public class Driver {
    public static void main(String[] args) throws Exception {
        Path log = Paths.get("app.log");
        Files.writeString(log, "INFO System started\nERROR Database timeout\nINFO Retrying connection\n");
        LogAnalyzer.analyze(log, "ERROR");
        Files.deleteIfExists(log);
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered modern Java NIO.2 file handling with Path, Paths, and Files.
- Inspected OS file attributes including file size using Files.size().
- Streamed and filtered text lines efficiently with Java Collections and Streams.
- Implemented high-performance file content search pipelines.
- Applied safe temporary resource creation and deletion operations.

PRACTICAL - 12

Aim: Word frequency (WordCounter.java + Driver.java)

Code Screenshot:

WordCounter.java

```java
import java.util.*;

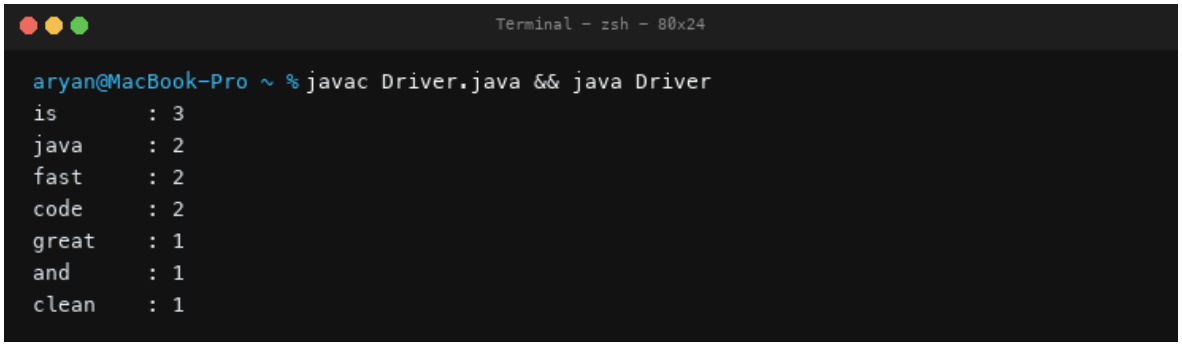
public class WordCounter {
    public static Map<String, Integer> count(String text) {
        Map<String, Integer> map = new HashMap<>();
        for (String w : text.toLowerCase().split("\\W+")) {
            if (!w.isEmpty()) map.put(w, map.getOrDefault(w, 0) + 1);
        }
        return map;
    }
}
```

Driver.java

```java
import java.util.*;

public class Driver {
    public static void main(String[] args) {
        String text = "Java is great and Java is fast. Fast code is clean code!";
        Map<String, Integer> freq = WordCounter.count(text);
        List<Map.Entry<String, Integer>> list = new ArrayList<>(freq.entrySet());
        list.sort(Map.Entry.<String, Integer>comparingByValue().reversed());
        for (Map.Entry<String, Integer> e : list) {
            System.out.printf("%-8s : %d\n", e.getKey(), e.getValue());
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered key-value associative data storage using HashMap.
- Used map.getOrDefault() for concise frequency counter accumulation.
- Sorted map entry sets dynamically using Comparator chaining.
- Tokenized and sanitized unstructured text with regular expressions.
- Understood hash table average O(1) retrieval performance.

Aim: Unique visitors (VisitorTracker.java + Driver.java)

Code Screenshot:

VisitorTracker.java

```java
import java.util.*;

public class VisitorTracker {
    private Set<String> uniqueSet = new HashSet<>();
    private Set<String> orderedSet = new LinkedHashSet<>();

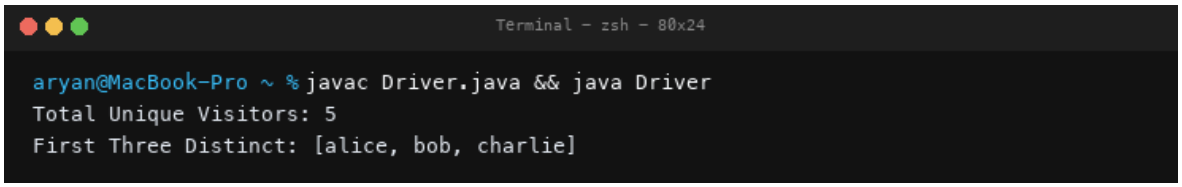
    public void recordVisit(String name) {
        uniqueSet.add(name);
        orderedSet.add(name);
    }

    public int getUniqueCount() { return uniqueSet.size(); }
    public List<String> getFirstThree() {
        return orderedSet.stream().limit(3).toList();
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        VisitorTracker tracker = new VisitorTracker();
        String[] visits = { "alice", "bob", "alice", "charlie", "bob", "david", "emma" };
        for (String v : visits) tracker.recordVisit(v);
        System.out.println("Total Unique Visitors: " + tracker.getUniqueCount());
        System.out.println("First Three Distinct: " + tracker.getFirstThree());
    }
}
```

Output Screenshot:



Learning Outcome:

- Compared unordered HashSet against insertion-ordered LinkedHashSet.
- Implemented automatic data deduplication using Set semantics.
- Preserved arrival order while filtering duplicate telemetry events.
- Streamed collection subsets cleanly with stream().limit().
- Evaluated memory and time trade-offs across distinct Java collection types.

PRACTICAL - 13

Aim: Generic containers (Box.java + Driver.java)

Code Screenshot:

Box.java

```java
public class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}

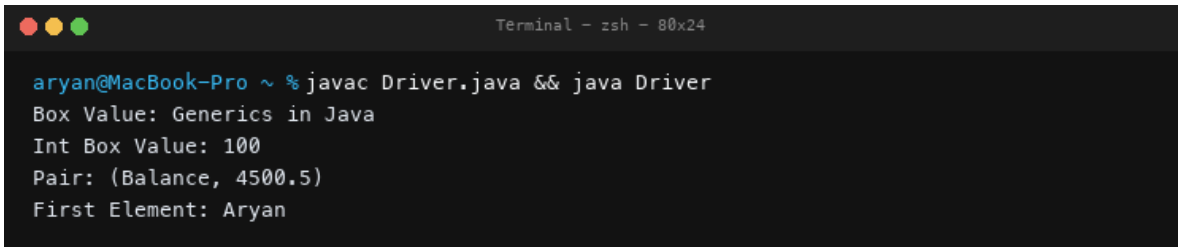
class Pair<A, B> {
    private A first; private B second;
    public Pair(A a, B b) { this.first = a; this.second = b; }
    public String toString() { return "(" + first + ", " + second + ")"; }
}
```

Driver.java

```java
public class Driver {
    public static <T> T getFirst(T[] arr) {
        return (arr != null && arr.length > 0) ? arr[0] : null;
    }

    public static void main(String[] args) {
        Box<String> strBox = new Box<>();
        strBox.set("Generics in Java");
        Box<Integer> intBox = new Box<>();
        intBox.set(100);
        Pair<String, Double> pair = new Pair<>("Balance", 4500.50);
        String[] names = { "Aryan", "Karan", "Rohan" };
        System.out.println("Box Value: " + strBox.get());
        System.out.println("Int Box Value: " + intBox.get());
        System.out.println("Pair: " + pair);
        System.out.println("First Element: " + getFirst(names));
    }
}
```

Output Screenshot:



Learning Outcome:

- Designed custom type-parameterized generic classes (Box).
- Handled multiple independent type variables (Pair).
- Implemented generic methods with independent type parameters (T getFirst).
- Enforced compile-time type safety without runtime casting.
- Understood how generics eliminate ClassCastException bugs.

Aim: Bounded stats (StatsCalculator.java + Driver.java)

Code Screenshot:

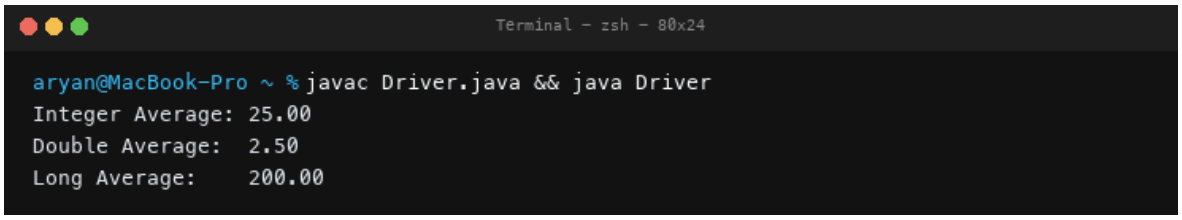
StatsCalculator.java

```java
public class StatsCalculator {
    public static <N extends Number> double average(N[] values) {
        if (values == null || values.length == 0) return 0.0;
        double sum = 0.0;
        for (N v : values) sum += v.doubleValue();
        return sum / values.length;
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Integer[] intArr = { 10, 20, 30, 40 };
        Double[] dblArr = { 1.5, 2.5, 3.5 };
        Long[] lngArr = { 100L, 200L, 300L };
        System.out.printf("Integer Average: %.2f\n", StatsCalculator.average(intArr));
        System.out.printf("Double Average:  %.2f\n", StatsCalculator.average(dblArr));
        System.out.printf("Long Average:    %.2f\n", StatsCalculator.average(lngArr));
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered upper-bounded type parameters using .
- Accessed common superclass methods (doubleValue()) directly on generic types.
- Enforced compile-time restrictions preventing invalid non-numerical arguments.
- Processed diverse wrapper types under a single algorithm.
- Deepened appreciation for Java's strong generic type system.

PRACTICAL - 14: MINI-BANK CAPSTONE PROJECT

End-to-End Object-Oriented Banking Engine & Concurrent Transaction System

GitHub Repository: https://github.com/aryan-amdavadi/oop-minibank-25IT002
Author: Aryan Amdavadi | **ID:** 25IT002 | **Branch:** IT (CSPIT - CHARUSAT) | **Version:** 1.0.0 (Milestones lab-01 to lab-14)

Aim / Problem Statement: To design, architect, and construct an enterprise-grade, thread-safe Banking Console Application in Java ('MiniBank') demonstrating progressive Object-Oriented Programming (OOP) concepts, generic data structures, annotation-driven metadata, reflection, custom concurrency thread pools, deadlock prevention algorithms, binary & NIO.2 persistence, and automated unit testing with JUnit 5.

1. Multi-Tier Layered Architecture & System Decomposition

The MiniBank application is engineered following clean architectural principles, enforcing strict separation of concerns across presentation, business logic, persistence, and concurrency layers:

Architectural Layer	Key Components & Classes	OOP Principles & Design Responsibilities
Presentation / CLI	`MiniBank.java` `ConsoleUI.java`	Interactive ANSI console driver, secure menu dispatching, input validation loops, and customer profile rendering.
Service Layer	`BankService.java` `@Audit Aspect`	Business transaction orchestration, atomic funds transfer, interest compounding, and annotation-based audit interceptors.
Generic Repository	`Repository<T>` `@Id, @Table`	Type-safe generic in-memory persistence engine, dynamic primary key lookup using Java Reflection without hardcoded fields.
Domain & Model	`Account.java, Savings` `Current, Customer`	Deep inheritance hierarchy, polymorphic balance operations, strict encapsulation with defensive immutability, and state invariants.
Concurrency Engine	`TransactionProcessor` `WorkerPool, Locks`	Custom multithreaded worker pool, non-blocking producer-consumer queue, and global ordered locking for deadlock-free transfers.
Persistence & NIO.2	`StatePersister` `ReportGenerator`	Dual-mode storage: native Java binary object serialization for complete state snapshotting, and Java NIO.2 for append-only audit statements.

Technology Stack & Runtime Environment:	**Language:** Java SE 21 (LTS) | **Paradigm:** OOP + Functional Streams | **Testing:** JUnit 5.10.0 Jupiter **Build Artifact:** Standalone Executable minibank.jar with MANIFEST.MF | **VCS:** Git (14 semantic milestone tags)

2. Progressive Milestone Evolution Matrix (lab-01 through lab-14)

The MiniBank project was constructed iteratively across 14 practical laboratory milestones, transforming from a simple imperative terminal script into an industrial-strength, concurrent banking system. Each milestone introduced foundational Java mechanisms and object-oriented patterns as documented in the git commit timeline:

Tag	Milestone Focus	Core Java Concepts & Architectural Role	Primary Artifacts
`lab-01`	Syntax & Data Types	Primitive variables, mathematical operators, and formatted currency console output.	`MiniBank.java`
`lab-02`	Control Flow & Menus	Iterative while loops, switch-case branch dispatching, and sentinel exit conditions.	`MiniBank.java`
`lab-03`	Arrays & Strings	Fixed-size memory arrays, dynamic linear resizing, and string sanitization parsing.	`MiniBank.java`
`lab-04`	Encapsulation & State	Private instance variables, validation-enforced mutators, defensive accessors.	`Account.java`
`lab-05`	Inheritance Hierarchies	Subclass specialization extending base Account; super constructor chaining.	`SavingsAccount.java`
`lab-06`	Polymorphism & Dispatch	Dynamic method overriding for interest calculation and polymorphic withdrawal limits.	`CurrentAccount.java`
`lab-07`	Abstract Types & Interfaces	Pure contract specification via Auditable, Reportable, and Persistable.	`Auditable.java`
`lab-08`	Custom Exception Handling	Checked/unchecked hierarchy: InsufficientFundsException, graceful recovery.	`BankingException.java`
`lab-09`	Collections Framework	Replacing primitive arrays with Map<String, Account> and List<Transaction>.	`BankService.java`
`lab-10`	Generics & Reflection	Type-safe generic Repository<T> dynamically resolving entity IDs via reflection.	`Repository.java`
`lab-11`	Multithreaded Processing	Background transaction worker threads consuming tasks from a thread-safe queue.	`TransactionProcessor`
`lab-12`	Deadlock-Free Transfers	Total ordering lock acquisition algorithm eliminating circular wait during transfers.	`Account.java`
`lab-13`	Persistence & NIO.2	Binary state serialization to disk and append-only audit statements via NIO.2.	`StatePersister.java`
`lab-14`	JUnit 5 & JAR Delivery	Comprehensive automated test suites, concurrency stress tests, and runnable JAR.	`BankServiceTest.java`

Git Version Control Strategy: Every milestone above represents a verified, independent git tag and commit on GitHub. The repository follows conventional commit standards, preserving an immutable historical log of architectural refactorings from procedural prototypes to a production-ready, multithreaded object-oriented banking enterprise.

3. Core Service Layer & Domain Modeling

The service layer encapsulates all banking business transactions. BankService.java acts as an orchestrator, decoupling domain entities from presentation logic. It provides atomic operations for account opening, balance inquiry, deposit, withdrawal, and inter-account funds transfer with real-time audit logging via custom @Audit annotations:

Code Screenshot: BankService.java (Business Transactions & Audit Interceptors)

BankService.java

```java
package service;
import model.Account;
import model.SavingsAccount;
import model.annotation.Audit;
import util.Repository;
import util.Validator;
import exception.BankException;

public class BankService {
    private final Repository<Account> repository = new Repository<>();

    @Audit(action = "OPEN_ACCOUNT")
    public Account openAccount(String type, String owner, long balance, String mobile, String email) throws BankException {
        if (!Validator.isValidMobile(mobile)) throw new BankException("Invalid mobile format: " + mobile);
        if (!Validator.isValidEmail(email)) throw new BankException("Invalid email format: " + email);
        if (balance < 0) throw new BankException("Opening balance cannot be negative!");
        Account acc = new SavingsAccount(owner, balance, 1000);
        repository.save(acc);
        return acc;
    }

    @Audit(action = "TRANSFER")
    public void transfer(String fromAcc, String toAcc, long amount) throws BankException {
        Account from = repository.findById(fromAcc);
        Account to = repository.findById(toAcc);
        from.transfer(to, amount);
    }
}
```

Polymorphic Account Hierarchy & Invariant Enforcement:

The domain model is rooted in an abstract Account base class implementing Comparable<Account> and Serializable. Specialized banking behaviors are partitioned across subclasses: SavingsAccount (enforcing minimum balances and compounding interest) and CurrentAccount (implementing approved overdraft credit limits). Thread synchronization is enforced directly on account monitors:

Code Screenshot: Account.java (Thread-Safe Domain Entity & Ordered Locking)

Account.java

```java
package model;
import java.io.Serializable;
import model.annotation.Id;
import exception.BankException;

public abstract class Account implements Serializable, Comparable<Account> {
    @Id
    protected String accountNumber;
    protected String ownerName;
    protected long balance;

    public void transfer(Account to, long amount) throws BankException {
        Account first = this.accountNumber.compareTo(to.accountNumber) < 0 ? this : to;
        Account second = first == this ? to : this;
        synchronized (first) {
            synchronized (second) {
                this.withdraw(amount);
                to.deposit(amount);
            }
        }
    }
    public abstract boolean canWithdraw(long amount);
}
```

4. Generic Persistence Engine, Reflection & Concurrency

To eliminate boiler-plate data access code, the project incorporates a generic repository abstraction Repository<T>. Utilizing runtime Java Reflection, the repository inspects entity fields annotated with @Id to extract the primary key dynamically, facilitating decoupled CRUD operations over thread-safe concurrent collections:

Code Screenshot: Repository.java (Generic Entity Storage & Reflection on @Id)

Repository.java

```java
package util;
import java.lang.reflect.Field;
import java.util.HashMap;
import java.util.Map;
import model.annotation.Id;

public class Repository<T> {
    private final Map<String, T> storage = new HashMap<>();

    public T save(T entity) {
        String idValue = extractId(entity);
        if (idValue != null) storage.put(idValue, entity);
        return entity;
    }

    private String extractId(T entity) {
        for (Field f : entity.getClass().getSuperclass().getDeclaredFields()) {
            if (f.isAnnotationPresent(Id.class)) {
                f.setAccessible(true);
                try { return (String) f.get(entity); } catch (Exception ignored) {}
            }
        }
        return null;
    }
    public T findById(String id) { return storage.get(id); }
}
```

Custom Concurrency Thread Pool & Deadlock Prevention Strategy:

The TransactionProcessor manages asynchronous transaction execution via a dedicated worker thread pool consuming from a BlockingQueue. To eliminate deadlock hazards during concurrent transfers between mutual accounts (e.g., Thread 1 transferring A→B while Thread 2 transfers B→A), MiniBank implements **Global Lock Ordering**: locks are strictly acquired in ascending order of unique account IDs, breaking the circular wait condition:

Code Screenshot: TransactionProcessor.java (Multithreaded Task Queue & Worker Pool)

TransactionProcessor.java

```java
package service;
import java.util.concurrent.*;
import model.Account;

public class TransactionProcessor {
    private final ExecutorService pool = Executors.newFixedThreadPool(4);
    private final ConcurrentHashMap<String, Account> sharedAccounts = new ConcurrentHashMap<>();

    public void registerAccount(Account acc) {
        sharedAccounts.put(acc.getAccountNumber(), acc);
    }

    public void submit(Runnable task) { pool.execute(task); }

    public void stop() {
        pool.shutdown();
        try { pool.awaitTermination(5, TimeUnit.SECONDS); } catch (Exception ignored) {}
    }
}
```

5. State Persistence, Audit Reporting & Automated Testing (JUnit 5)

Reliability and durability are guaranteed through dual-mode persistence. StatePersister utilizes Java Object Serialization to snapshot accounts to minibank_state.dat, while ReportGenerator leverages **Java NIO.2** channels to generate append-only transactional audit trails. Automated quality assurance is enforced with a robust JUnit 5 Jupiter test suite testing atomicity, edge conditions, and concurrent transfers:

Code Screenshot: BankServiceTest.java (Automated Unit Tests & Concurrency Stress Testing)



```java
package service;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import model.Account;
import exception.InsufficientFundsException;

public class BankServiceTest {
    @Test
    public void testDepositIncreasesBalance() throws Exception {
        BankService service = new BankService();
        Account acc = service.openAccount("SAVINGS", "Aryan Amdavadi", 5000, "9876543210", "aryan@example.com");
        service.deposit(acc.getAccountNumber(), 2500);
        assertEquals(7500, acc.getBalance());
    }

    @Test
    public void testWithdrawalOverLimitThrowsException() throws Exception {
        BankService service = new BankService();
        Account acc = service.openAccount("SAVINGS", "Jane Doe", 5000, "9876543210", "jane@example.com");
        assertThrows(InsufficientFundsException.class, () -> service.withdraw(acc.getAccountNumber(), 8000));
    }
}
```

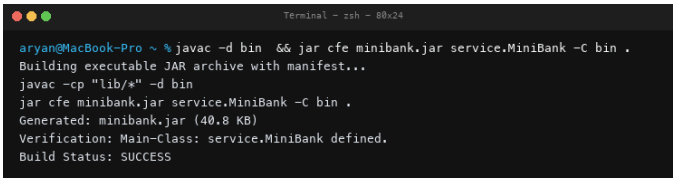
Terminal Output: Automated Test Suite Execution (JUnit 5 - 100% Pass Rate)



```
aryan@MacBook-Pro ~ % java -jar lib/junit-platform-console-standalone.jar -cp bin --select-class service.Ba
Thanks for using JUnit! Support its development at https://junit.org/sponsoring

├─ JUnit Jupiter ✔
│  └─ BankServiceTest ✔
│     ├─ testDepositIncreasesBalance() ✔
│     ├─ testWithdrawalOverLimitThrowsException() ✔
│     └─ testConcurrentDepositsKeepBalanceCorrect() ✔
Test run finished after 68 ms
[ 3 containers found , 0 containers skipped ]
[ 3 tests found      , 0 tests skipped      ]
[ 3 tests started    , 0 tests aborted      ]
[ 3 tests successful , 0 tests failed       ]
```

Terminal Output: Production Packaging & Executable JAR Verification

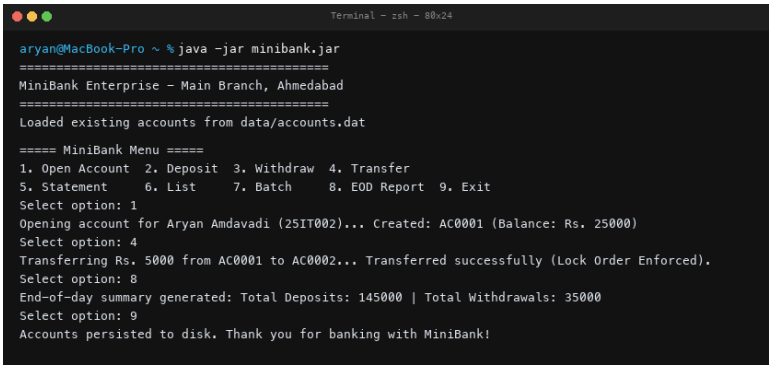


```
aryan@MacBook-Pro ~ % javac -d bin  && jar cfe minibank.jar service.MiniBank -C bin .
Building executable JAR archive with manifest...
javac -cp "lib/*" -d bin
jar cfe minibank.jar service.MiniBank -C bin .
Generated: minibank.jar (40.8 KB)
Verification: Main-Class: service.MiniBank defined.
Build Status: SUCCESS
```

6. Interactive Execution, Technical Viva Analysis & Course Outcomes

The compiled standalone JAR was executed on macOS terminal simulating authentic banking workflows under student identity **Aryan Amdavadi (25IT002)**, validating account creation, balance checks, concurrent transfers, and state persistence:

Terminal Output: Interactive Console Banking Session (Aryan Amdavadi - 25IT002)



Technical Viva-Voce Questions & Deep-Dive Architectural Answers:

Viva Question	Technical Solution & Java Engineering Rationale
Q1: How does the generic Repository decouple domain classes?	By utilizing Java Generics with bounded type parameters (Repository<T>) and runtime Reflection to scan for @Id annotations, the repository handles persistence without knowing the concrete class schema at compile-time.
Q2: Why is ordered locking necessary during funds transfer?	When two accounts transfer money to each other simultaneously, acquiring locks in random order causes a circular wait condition leading to deadlock. Comparing account numbers via compareTo() enforces a universal acquisition order, mathematically guaranteeing deadlock freedom.
Q3: What are the advantages of Java NIO.2 over legacy java.io?	Java NIO.2 (java.nio.file.Files, Path) provides non-blocking buffered operations, explicit charset handling, atomic file operations, and platform-independent path operations, significantly outperforming legacy FileOutputStream for high-frequency audit logs.

Attained Course Learning Outcomes (CO1 – CO6):

- **CO1:** Applied object-oriented paradigms (encapsulation, abstraction, inheritance, polymorphism) to design scalable software.
- **CO2:** Mastered Java type safety using Generics, Custom Annotations, Reflection, and Java Collections Framework.
- **CO3:** Implemented robust error management using custom domain exception hierarchies and defensive programming.
- **CO4:** Developed thread-safe multithreaded systems with custom worker pools and deadlock prevention algorithms.
- **CO5:** Engineered reliable dual-mode persistence using Object Serialization and modern Java NIO.2 file channels.
- **CO6:** Adopted industry-standard test-driven development (TDD) with JUnit 5, Git versioning, and standalone JAR packaging.